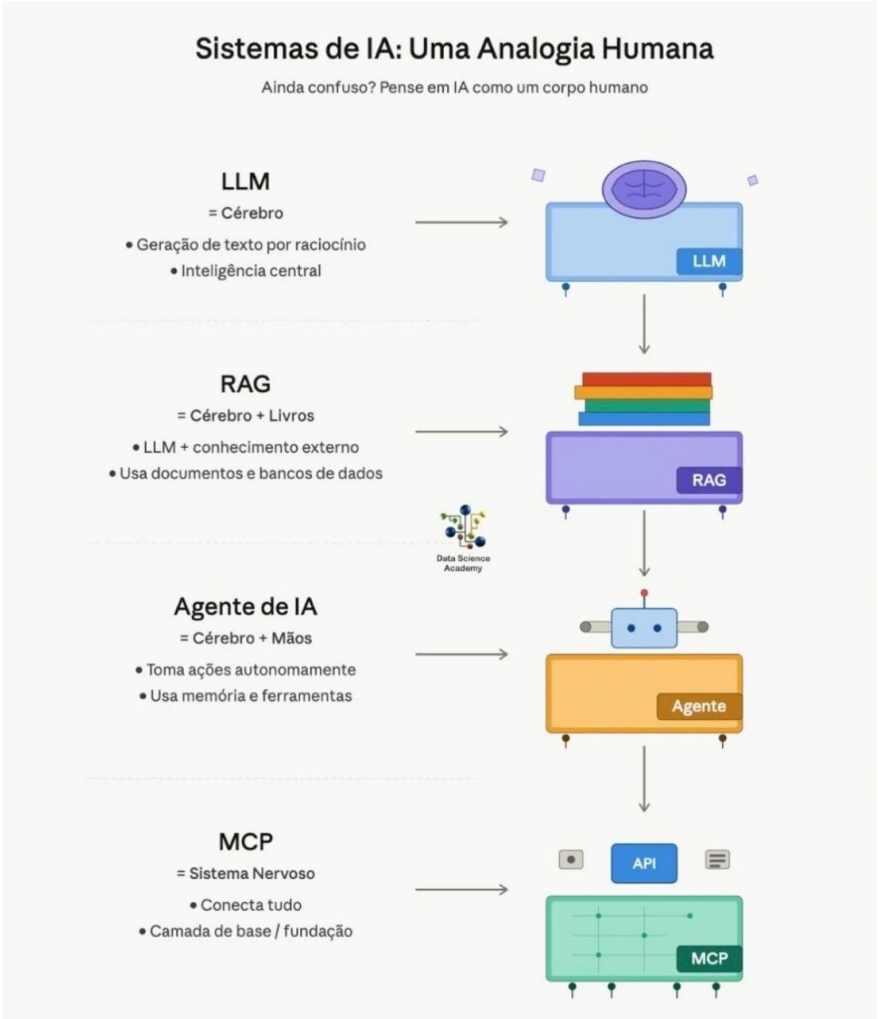
# IA Generativa: LLM, RAGs,

Tecnologias Inteligentes Aplicadas à Saúde

Alexandre Zamberlan




Fonte: Renato Augusto

https://www.youtube.com/@RenatoAugustoTech

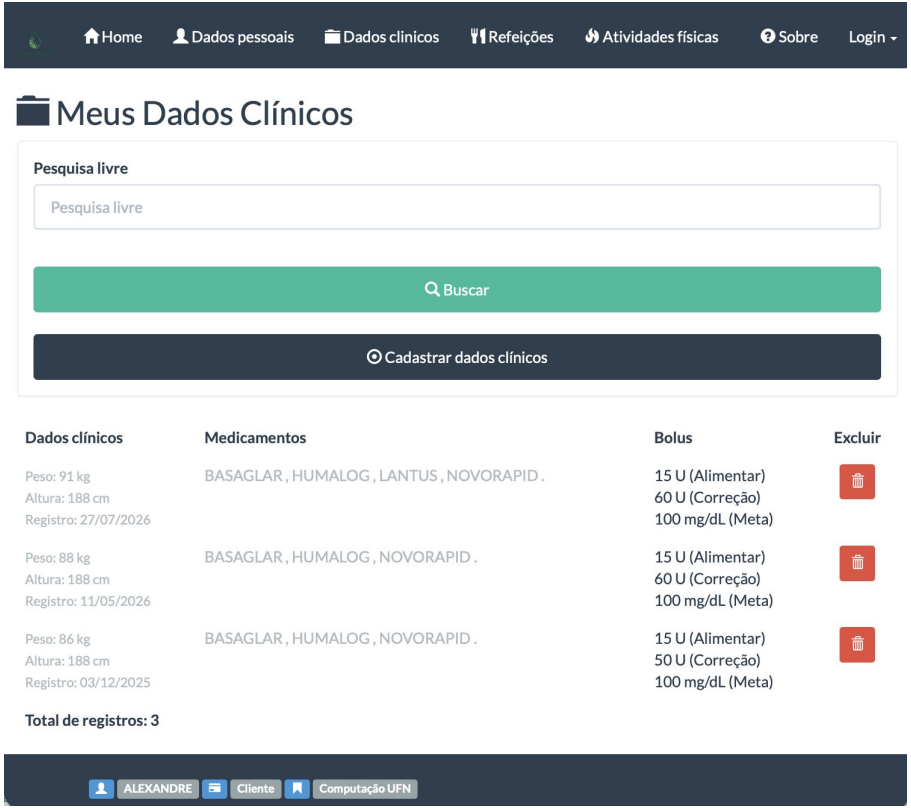
# Exemplo prático: SOMDiabetes | Pipeline recomendar insulina

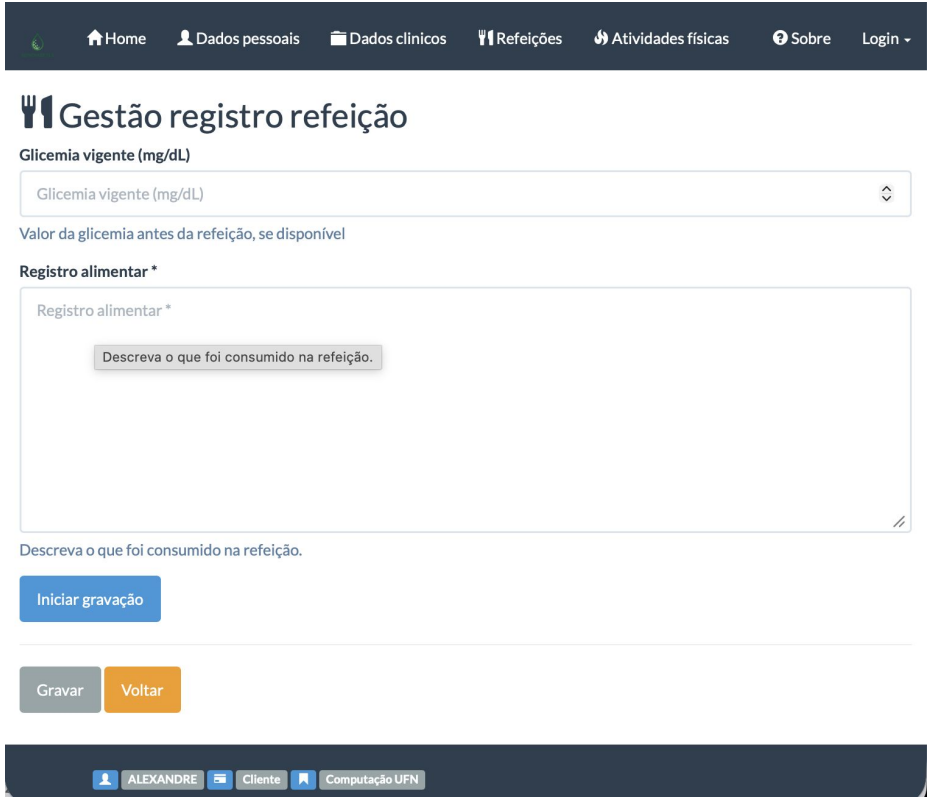


# Exemplo prático: SOMDiabetes

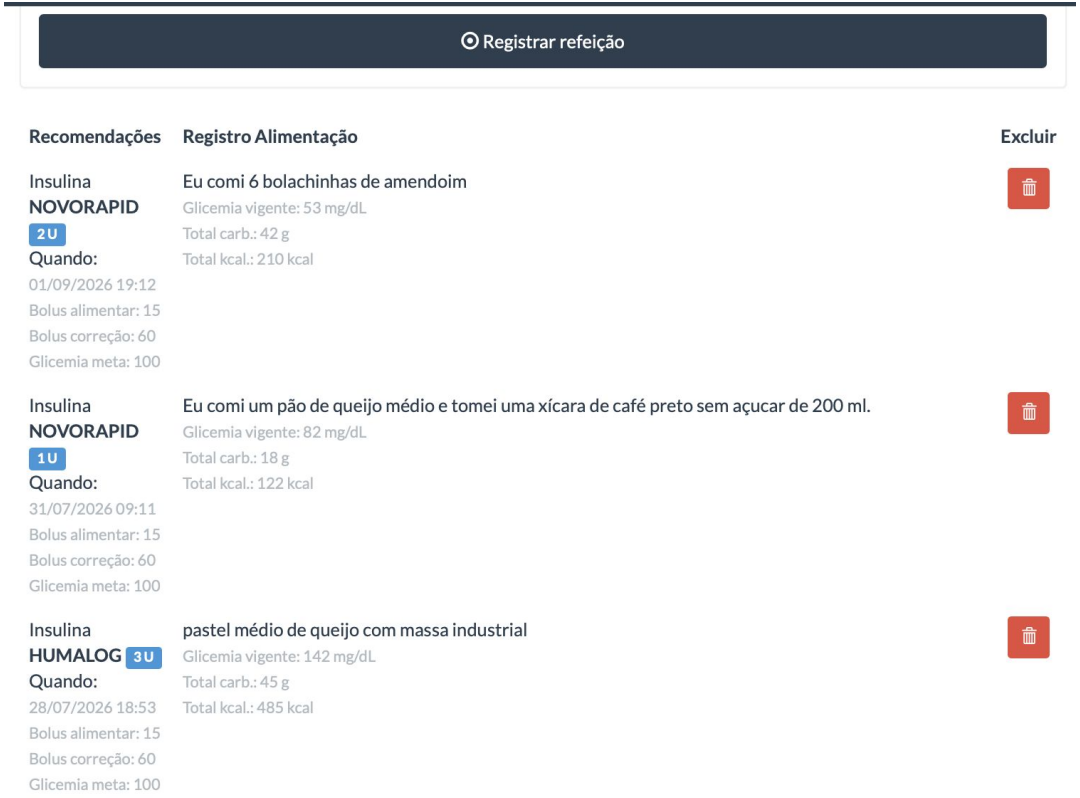


# Exemplo prático: SOMDiabetes | Entrada



# Exemplo prático: SOMDiabetes | Saída



# Exemplo prático: SOMDiabetes | Gravando refeição

```
class RegistroRefeicao(models.Model):
    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = gerar_hash()

        medicamentos = [med.nome_comercial for med in
                        self.cliente.medicamentos.all()] if self.cliente.medicamentos.exists() else []
        tipo_diabetes = self.cliente.tipo_diabetes or "SEM DIABETES"
        bolus_alimentar = self.cliente.bolus_alimentar or 1
        bolus_correcao = self.cliente.bolus_correcao or 1
        glicemia_meta = self.cliente.glicemia_meta or 100
        glicemia_atual = self.glicemia_vigente
        descricao_alimentacao = self.registro_alimentacao

        contexto_json = Conecta.montar_json(medicamentos, tipo_diabetes, bolus_alimentar, bolus_correcao, glicemia_meta,
                                            glicemia_atual, descricao_alimentacao)
        total_tokens = 0
        resposta_json, total_tokens = Conecta.gerar_recomendacoes(contexto_json)

        if resposta_json and resposta_json.strip():
            try:
                lista_alimentos, carboidratos, calorias, qtd_insulina, nome_insulina = Conecta.desmontar_json(
                    resposta_json)
                self.total_carboidratos = int(carboidratos)
                self.total_calorias = int(calorias)
                self.quantidade_insulina_recomendada = int(qtd_insulina)
                self.nome_insulina = nome_insulina
            except (ValueError, KeyError, json.JSONDecodeError) as e:
                print(f"Erro ao processar resposta da IA: {e}")
                self.total_carboidratos = 0
                self.total_calorias = 0
                self.quantidade_insulina_recomendada = 0
                self.nome_insulina = "Não disponível"
        else:
            self.total_carboidratos = 0
            self.total_calorias = 0
            self.quantidade_insulina_recomendada = 0
            self.nome_insulina = "Não disponível"

        tokens_string = str(total_tokens).split(' ')
        self.quantidade_tokens_consumidos = int(tokens_string[1]) if len(tokens_string) > 1 else 0

        super(RegistroRefeicao, self).save(*args, **kwargs)
```

# Exemplo prático: SOMDiabetes | Recomendando insulina

```
15  class Conecta:
368     @staticmethod
369     def gerar_recomendacoes(contexto_json):
370         try:
371             client = Conecta.conectar_api()
372             if isinstance(client, str):
373                 return client
374
375             papel_esperado_ia = "Nutricionista especializada em contagem de carboidratos, calorias e cálculo de insulina."
376
377             resposta_desejada = "Retorne SOMENTE um JSON com um único objeto com o nome dos alimentos, quantidade de carboidrato, quantidade
                de caloria, quantidade de insulina necessária e o nome da insulina a ser utilizada."
378
379             exemplo_json = '{"lista_alimentos": "", "total_carboidratos": , "total_calorias": , "quantidade_insulina_recomendada": ,
                "nome_insulina": ""}'
380
381             prompt = f"""
382                 Papel: {papel_esperado_ia}
383
384                 Contexto: {contexto_json}
385
386                 Resposta: {resposta_desejada}
387
388                 Exemplo de JSON esperado: {exemplo_json}
389                 """
390
391             response = client.generate_content(prompt)
392
393             resposta_json = response.candidates[0].content.parts[0].text.strip().strip('```json').strip('```')
394             total_tokens = client.count_tokens(contents=prompt + resposta_json)
395
396             return resposta_json, total_tokens
397
398         except Exception as e:
399             print(f"Erro ao gerar recomendações: {str(e)}")
400             erro = f"Não foi possível conectar no servidor para gerar recomendações. Por favor, tente novamente mais tarde. Erro: {str(e)}"
401             return erro, 0
```

# Exemplo prático: SOMDiabetes | Classe Conecta

```
16  class Conecta:
17      todos_tokens = 0
18
19      @staticmethod
20      def testar_conexao(model):…
28
29      @staticmethod
30      def conectar_api():  # conecta com api gemini…
64
65      ####### PARTE DO ESPECIALISTA EM NUTRICAO #######
66
67      @staticmethod
68      def montar_json(medicamentos, tipo_diabetes, bolus_alimentar, bolus_correcao, glicemia_meta, glicemia_atual,…
83
84      @staticmethod
85      def desmontar_json(resposta_json):…
97
98      @staticmethod
99      def gerar_recomendacoes(contexto_json):…
132
```

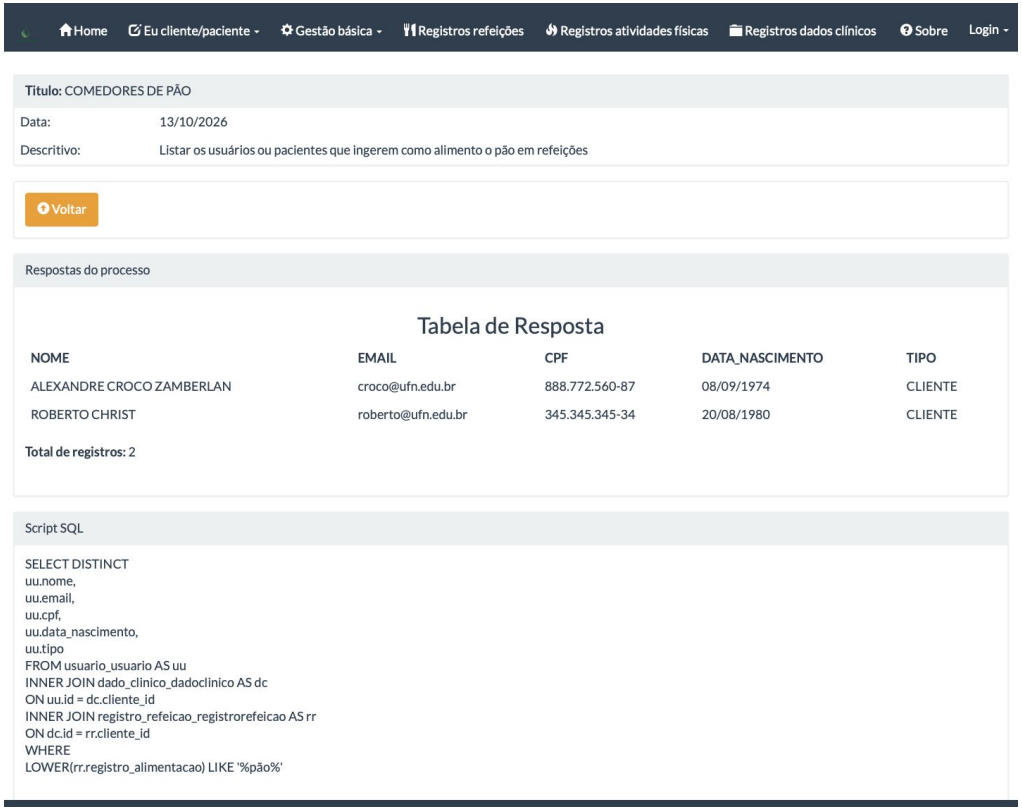
# Exemplo prático: SOMDiabetes | Classe Conecta

```
134     ####### PARTE DO ESPECIALISTA EM SQL #######
135     @staticmethod
136     def houve_alteracao_banco():…
159
160     @staticmethod
161     def atualizar_contexto_json():…
171
172     @staticmethod
173     def atualizar_contexto_toon():…
183
184     @staticmethod
185     def carregar_schema_json():…
196
197     @staticmethod
198     def carregar_schema_toon():…
209
210     @staticmethod
211     def gerar_sql(pergunta):…
279
280     @staticmethod
281     def checar_consulta_segura(sql):  # NAO ALTERAR, REUTILIZAR…
301
302     @staticmethod
303     def executar_sql(sql):  # NAO ALTERAR, REUTILIZAR…
349
350     @staticmethod
351     def filtrar_resultados(resultados, nomes_campos):…
373
374     @staticmethod
375     def gerar_tabela_html(nomes_campos, lista_dados):…
```

# Exemplo prático: SOMDiabetes | Pipeline gerar consulta



# Exemplo prático: SOMDiabetes | Pipeline gerar consulta



# Exemplo prático: SOMDiabetes | Pipeline gerar consulta

```python
    @staticmethod
    def carregar_schema_json():
        # Carrega o esquema do banco de dados do arquivo schema.json
        try:
            schema_path = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), 'schema.json')

            with open(schema_path, "r", encoding='utf-8') as file:
                schema_data = json.load(file)
                return schema_data
        except Exception as e:
            print(f"Erro ao carregar schema: {str(e)}")
            return None

    @staticmethod
    def carregar_schema_toon():
        # Carrega o esquema do banco de dados do arquivo schema.toon
        try:
            schema_path = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))), 'schema.toon')

            with open(schema_path, "r", encoding='utf-8') as file:
                schema_string = file.read()
                return schema_string
        except Exception as e:
            print(f"Erro ao carregar schema: {str(e)}")
            return None
```

```python
    @staticmethod
    def gerar_sql(pergunta):
        try:
            model = Conecta.conectar_api()  # conecta com gemini api
            if isinstance(model, str):  # Se retornou erro
                return model
            # Carrega o schema do banco de dados
            schema_data = Conecta.carregar_schema_toon()
            if not schema_data:
                return "Erro ao carregar o esquema do banco de dados."
            # Constrói o prompt com o contexto do schema
            context_prompt = f"""
            Você é um especialista em SQL para Django. Gere UMA consulta SQL válida baseada nas tabelas disponíveis.

            O Schema é:
            {schema_data}

            REGRAS CRÍTICAS:
            1. Retorne APENAS UMA instrução SELECT válida
            2. NÃO use ponto e vírgula (;)
            3. FAÇA SOMENTE SELEÇÃO de dados (SELECT). NÃO use INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, EXEC, MERGE ou
            TRUNCATE
            4. Para buscar por nome ou campos descritivos use: WHERE campo LIKE '%parte_do_texto%'
            5. SEMPRE selecione campos completos (ex: u.nome, não SUBSTRING ou similar)
            6. NÃO use funções que dividem strings em caracteres
            7. Máximo 5 colunas no SELECT
            8. Ignore campos padroes do django
            9. Trate dados com letras maiusculas e minusculas igualmente (case insensitive)
            10. Sinônimos comuns: usuario = participante = estudante = aluno; participação = frequência = presença;

            PERGUNTA: {pergunta}
            SQL:
            """

            response = model.generate_content(context_prompt)
            tokens_lista = str(model.count_tokens(contents=context_prompt + response.text)).split(' ')
            Conecta.todos_tokens += int(tokens_lista[1])
            print("Conectado ao Gemini 2.5 Flash para aplicar consulta.",
                  f"Total tokens usados: {Conecta.todos_tokens}")
            sql_resposta = response.text.strip()
```

# Exemplo prático: SOMDiabetes | Pipeline gerar consulta

```
250             # Remove possíveis marcadores de código markdown
251             if sql_resposta.startswith('```sql'):
252                 sql_resposta = sql_resposta[6:]
253             if sql_resposta.startswith('```'):
254                 sql_resposta = sql_resposta[3:]
255             if sql_resposta.endswith('```'):
256                 sql_resposta = sql_resposta[:-3]
257
258             # Remove ponto e vírgula no final e quebras de linha extras
259             sql_limpo = sql_resposta.strip()
260             if sql_limpo.endswith(';'):
261                 sql_limpo = sql_limpo[:-1]
262
263             # Se houver múltiplas instruções separadas por ';', pega apenas a primeira
264             if ';' in sql_limpo:
265                 sql_limpo = sql_limpo.split(';')[0].strip()
266
267             return sql_limpo
268         except Exception as e:
269             erro = f"Não foi possível conectar no servidor para gerar a consulta. Por favor, tente novamente mais tarde.
                Erro: {str(e)}"
270             return erro
271
```

# Exemplo prático: SOMDiabetes | Pipeline gerar consulta

```
295     def executar_sql(sql):  # NAO ALTERAR, REUTILIZAR
296         try:
297             # Verifica se a consulta é segura
298             if not Conecta.checar_consulta_segura(sql):
299                 return "Uma consulta potencialmente insegura foi detectada. Por favor, tente novamente."
300 
301             # Conecta no banco com cursor e obtém os resultados
302             with connection.cursor() as cursor:
303                 cursor.execute(sql)
304                 resultados = cursor.fetchall()
305 
306                 # Se não houver resultados, retorna a mensagem
307                 if not resultados:
308                     return "Nenhum resultado relevante foi encontrado."
309 
310                 # Obtém os nomes dos campos
311                 nomes_campos = [i[0].upper() for i in cursor.description]
312 
313             # Verifica se resultados é string (mensagem de erro)
314             if isinstance(resultados, str):
315                 return resultados
316 
317             # Se não houver "order by" na consulta, ordena a lista de listas (com proteção)
318             if "order by" not in sql.lower() and resultados:
319                 try:
320                     # Verifica se os resultados têm pelo menos um elemento para ordenar
321                     if len(resultados) > 0 and len(resultados[0]) > 0:
322                         lista_dados = sorted(resultados, key=lambda x: x[0] if x and len(x) > 0 else '')
323                     else:
324                         lista_dados = resultados
325                 except (IndexError, TypeError, AttributeError):
326                     lista_dados = resultados
327             else:
328                 lista_dados = resultados
329 
330             # Filtra os resultados
331             lista_filtrada = Conecta.filtrar_resultados(lista_dados, nomes_campos)
332 
333             if lista_filtrada == 'Erro':
334                 raise Exception("Mais de 5 colunas retornadas. Por favor, refine sua consulta.")
335 
336             # Gera a tabela HTML com os resultados filtrados
337             return Conecta.gerar_tabela_html(nomes_campos, lista_filtrada)
```